IOT6000U – Digital Innovation and Product Development

Week 2 Lab - Modernising a Legacy Game AI

Scenario

A games company has found an old **Connect Four** program in its archive. It is written in **Java**, nobody who built it is still around, and there is no documentation. All anyone knows is that the computer opponent uses **"some kind of search"**.

You have just learned **minimax** and **alpha-beta pruning**, so the company has asked your team to take it on. They want the game **in Python**, and they want to be confident that the algorithm is correct and has been converted correctly. After that, they would like some new features.

Lab focus: understanding unfamiliar code and connecting it to what you learned in the lecture.

How to work

- Work in mixed teams, ideally combining apprentices from different pathways.
- You are welcome to use AI tools. If you do, your team owns the result: make sure you understand it and that it works well.

Getting started: run the Java version

Download the code from Moodle and unzip it. Keep this layout:

- `src/` - the Java source code (packages `algorithm` and `ui`)
- `images/` - the disc images used by the game

Keep the images folder in the top folder, next to src, not inside it. The game looks for the images in the folder it is run from. If it can't find them, it still runs but the discs are invisible.

Option A - any IDE (IntelliJ IDEA, Eclipse, VS Code with the Java extensions, etc.): open the top folder as a project, mark `src` as the source folder if asked, and run `Main` in `src/ui/Main.java`.

Option B - an installed JDK (any recent version, e.g. 17 or 21): open a terminal in the top folder and run:

```
javac -d out -sourcepath src src/ui/Main.java
java -cp out ui.Main
```

This works the same on Windows, macOS and Linux. Make sure the terminal is in the top folder (the one containing src and images). You may see deprecation warnings when compiling; they are safe to ignore.

Your tasks

1. **Play:** Run the Java version and play a few games against it. How well does it play?
2. **Read:** Work out how the AI chooses its moves, and how that relates to minimax and alpha-beta pruning.
3. **Port:** Convert the game to Python as a desktop app. Make sure the algorithm is correct and that your version behaves like the original.
4. **Add difficulty:** Let players choose between easy, medium and hard.
5. **Going further (optional):** Add features of your own. For example, make the AI more efficient (such as move ordering: trying the most promising moves first, so alpha-beta can prune more), or add new features such as choosing who goes first, an undo button, hints for the player, a score counter, or AI-versus-AI games.

By the end of the lab

By the end of the lab, you should understand how the AI works and have made at least one change to it, such as adding difficulty levels. Some teams may also complete the full conversion to Python.